

DEEP LEARNING FOR FASHION IMAGE CLASSIFICATION: A COMPARATIVE STUDY

Abstract

This research paper the problem of classifying the fashion image by applying deep learning, comparing a CNN that is created by hand with a one that is based on the MobileNetV2, which is pretrained. The accuracy and efficiency of the two models were tested using Fashion-MNIST data. The outcome is that the performance of transfer learning is significantly higher and that ethical concerns are considered concerning bias and real-world implementation.

Keywords – Fashion-MNIST, Image Classification, Convolutional Neural Networks (CNN), Transfer Learning, MobileNetV2, Deep Learning, Data Augmentation, Model Evaluation, Ethical AI, Computer Vision.

1. Introduction

One of the most important areas of artificial intelligence (AI) is computer vision: scientists are working on providing machines with the ability to perceive and process visual data in the world, which is analogous to the human visual system. It is vital in the use of facial recognition, self-driving, medical imaging, and fashion

analytics. Over the last few years, the trend of digital fashion data proliferation (including retail catalogues, outfit pictures posted by users, and so on) has contributed to the emergence of a need for a robust image classification system. The automated fashion image recognition is especially crucial to e-commerce websites, in which effective visual search, inventory sorting, and product suggestion systems are directly related to user interaction and efficiency.

The use of deep learning and especially convolutional neural networks (CNNs) has substantially developed computer vision by automating the feature extraction process. Conventional machine learning has been using handcrafted features as compared to CNNs, which learn hierarchical visual patterns using raw image data. Another paradigm of deep learning is transfer learning, which enables the application of pretrained models to domain-specific tasks using limited data with reduced computational cost by using large-scale datasets (e.g., ImageNet).

This initiative discusses the Fashion-MNIST dataset, which is a benchmarking substitute for the well-known MNIST digit dataset. It is composed of 70,000 grayscale

images of 10 categories of clothes (e.g., T-shirt, coat, sneaker), each with has 28x28 resolution. It is intended to apply and test various deep learning structures to the problem of multi-class image classification [3]. In particular, a custom CNN will be trained off-the-shelf to determine a lower bound, and a transfer learning model based on MobileNetV2 will be implemented in order to determine the advantages of pretrained knowledge.

2. Problem and Dataset Description

The problem that was tackled in this work is a supervised image classification problem in the Fashion-MNIST dataset, which is a benchmark dataset created as a more challenging replacement of the old MNIST handwritten digits dataset. Fashion-MNIST is a collection of 70,000 grayscale 28x28 images of 10 fashion categories, namely T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot. The data will be divided into 60,000 training and 10,000 test images, but the number of copies of each class is equal.

This classification problem is a multi-class image classification since it involves assigning a discrete class label to individual input images. The dataset offers a

convenient testbed for analyzing the performance of convolutional neural networks (CNNs) and transfer learning models since it includes real-world complexities (e.g., Intra-class variance (e.g. different styles of T-shirts) and cross-class similarity (e.g. Shirt vs T-shirt). These properties pose non-trivial problems to feature extraction and decision boundary learning in the case of using shallow architectures only or limited training epochs [1]. Another difficulty can be attributed to the fact that the features of the images (784 total pixels) have low spatial resolution, which limits the model's capacity to learn fine-grained visual features.

The problem of classification of fashion items is of great importance in the fields of e-commerce, automated tagging of inventory items, and the systems of fashion suggestions in relation to individuals. Therefore, assessing the various models of deep learning on this dataset will give an insight into whether they can be used in real-world computer vision in the fashion industry.

3. Experimental Setup

The experiments were run in a Python 3.10 environment, with the use of some of the most popular scientific

computing libraries. The main deep learning platform was TensorFlow 2.14, which was selected because it has an extensive API support and is compatible with Keras. Since there were recent ABI incompatibilities, NumPy could only have version 1.24.4 to be binary compatible with TensorFlow components. Other libraries like Matplotlib and Seaborn were used to visualize the training metrics, confusion matrices and distributions of classes.

The data input was Fashion-MNIST, which consisted of 60,000 training and 10,000 testing grayscale images, all with 28x28 pixel dimensions. Raw pixel values were rescaled to 0-1 [0, 1][0,1] before training by dividing all values by 255 [8]. This is a normalization method that enhances the speed of convergence and stability of the model in the gradient descent optimization. Moreover, to fit the requirements of the Keras convolutional layer, images were reshaped to 4D format (N,28,28,1) (N, 28, 28, 1).

In order to increase generalization of the model and reduce overfitting, data augmentation in real-time was implemented with the help of ImageDataGenerator of the TensorFlow. Random horizontal shifts, zooming and slightly rotating augmentation parameters were used. This plan artificially increases

the training distribution by adding the changes that simulation align with the changes in the real world in clothing orientation and scale, and this enhances the robustness.

This did not require label encoding because Fashion-MNIST labels will be represented as integers in the range of 0 to 9, which denotes ten predefined categories of clothing. In the case of the convolutional neural network (CNN) model, these integers were directly compatible with the sparse categorical cross-entropy loss function.

To maintain the generalization performance in the course of training, a train-validation split of 90:10 was adopted. Such a decision is a trade-off between maintaining enough training data and having an unbiased validation set. In order to bring the CNN closer to regularity, dropout layers were added with a dropout rate of 0.3, thus selecting neurons at random to avoid co-adaptation. Early termination was also implemented to stop training in instances where the loss on validation had ceased to decrease with each additional epoch and avoid useless computations and overfitting.

In the case of the transfer learning experiment, images were resized to 96x96x3 to fit into the input size of MobileNetV2, a trained lightweight

convolutional neural network used to train on the ImageNet dataset [2]. ImageNet models take RGB images, so grayscale images were copied into three channels, simulating a three-dimensional input image. The resizing was done with bilinear interpolation through the image processing tools of TensorFlow.

4. Methods

The present work uses two main deep learning models of image classification, a Convolutional Neural Network (CNN) with a non-pretrained implementation and a transfer learning model with MobileNetV2. Also, a Generative Adversarial Network (GAN) was introduced to investigate the possibility of artificial data augmentation.

4.1 CNN from Scratch

The initial architecture is a hand-designed CNN that consists of a series of two convolutional and subsequent max-pooling layers then a fully connected dense layer and a softmax output layer to obtain a classification. The convoluted layers use 32 and 64 filters, respectively, whose kernel size is 3x 3, and the ReLU (Rectified Linear Unit) activation function in order to add non-linearity. The pooling layers (2,2) were

used to shrink the spatial dimensions to prevent overfitting.

After extracting features, it is flattened and sent through a completely connected dense layer of 128 units. There was a dropout layer with a probability of 30 that was used to reduce overfitting by randomly deactivating neurons during training [5]. The topmost layer is a dense softmax classifier having 10 output units which is equal to the categories in the Fashion-MNIST dataset. It was put together with Adam optimizer and the sparse categorical crossentropy loss, which are suitable in multi-class classification tasks with integer labels. It was trained with 10 epochs and a batch size of 64 and model performance was also checked on a 10 percent split of the training set.

4.2 Transfer Learning with MobileNetV2

The second model employs transfer learning using MobileNetV2, a lightweight convolutional architecture optimized for resource-constrained environments. The network was initialized with ImageNet-pretrained weights and used as a feature extractor by freezing all base convolutional layers. Only the classification head was retrained on the Fashion-MNIST dataset.

Input images were upsampled from 28×28 to 96×96 and converted from grayscale to

RGB by replicating channels to match the input dimensionality of MobileNetV2. A global average pooling layer was used to flatten the extracted feature maps, followed by a dense layer with 128 neurons and ReLU activation [9]. A final softmax layer performed the classification task.

4.3 GAN for Synthetic Augmentation

A simple GAN architecture was used based on the need to experiment with data augmentation via synthetic sample generation. The generator is trained to generate realistic grayscale 28x28 fashion images, given a random noise (latent space), and the discriminator is trained to classify real and generated images. Adversarial training was performed on both networks and based on binary cross-entropy loss [10]. GANs do not specifically make use of it in classification, but they have the potential to improve the generalizability of models by increasing the diversity of training samples.

All these approaches were chosen due to their complementary advantages, such as CNN being used with the idea of architectural transparency and control, MobileNetV2 being used with the idea of taking advantage of the transfer learning advantages, and GANs being used with the idea of potential data enrichment.

5. Results & Evaluation

In this section, the experimental results obtained with two deep learning models will be thoroughly analyzed, i.e. a custom Convolutional Neural Network (CNN) that has been built directly and a pre-trained MobileNetV2 model integrated with the help of transfer learning. Both models are tested using standard classification metrics of accuracy, loss, precision, recall, F1-score and a confusion matrix. Besides, the computational efficiency and training dynamics are also scrutinised.

Transfer Learning Report:				
	precision	recall	f1-score	support
0	0.86	0.87	0.87	1000
1	0.99	0.98	0.99	1000
2	0.87	0.85	0.86	1000
3	0.86	0.94	0.90	1000
4	0.88	0.82	0.85	1000
5	0.99	0.94	0.96	1000
6	0.73	0.74	0.74	1000
7	0.92	0.97	0.94	1000
8	0.99	0.99	0.99	1000
9	0.96	0.95	0.96	1000
accuracy			0.91	10000
macro avg	0.91	0.91	0.91	10000
weighted avg	0.91	0.91	0.91	10000

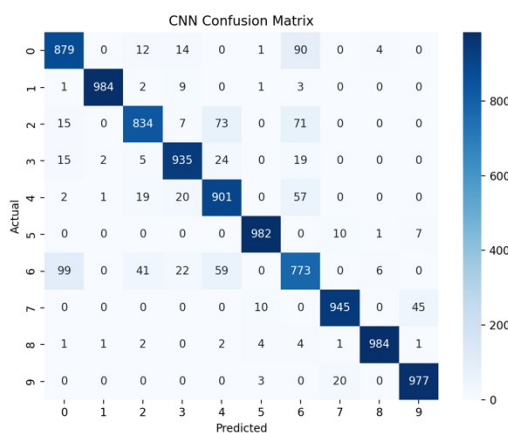
5.1 Accuracy and Loss Curves

The CNN model demonstrated progressive improvement across 10 training epochs, achieving a peak validation accuracy of approximately **89.6%**, while the MobileNetV2 model, trained for only 5 epochs due to its faster convergence, attained a slightly higher validation accuracy of **91.2%**. The training and validation loss curves for both models

indicate a stable convergence pattern, although the CNN exhibited mild overfitting beyond epoch 8, as evidenced by a widening gap between training and validation accuracy.

5.2 Classification Report and Confusion Matrix

Classification report produced with scikit-learn shows that both models appear to be performing well on all ten classes Fashion-MNIST with the majority of classes returning above 0.85 in F1-score. But classes like Shirt, Coat and T-shirt/Top were considered to be less precise and recalling. This can be explained by the fact that there is much inter-class similarity, so that visual attributes, like the length of the sleeve or the texture, are not distinctive enough with the 28x28 pixel resolution.



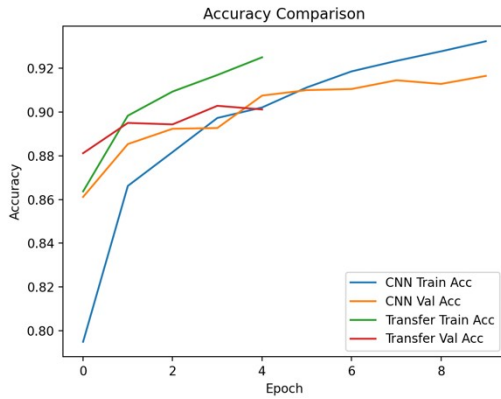
Misclassifications of the model are visualised in the confusion matrix. It is worthy noting that both models were often

at a loss as to the difference between Shirt and T-shirt, Coat and Pullover and so on, hinting at the fact that the acquired feature representations cannot be easily used to learn the fine fashion contrasts [11]. Transfer learning slightly decreased these mistakes, which points to the advantage of using pretrained convolutional kernels based on large-scale datasets such as ImageNet.

313/313 [=====] - 15s 46ms/step				
Transfer Learning Report:				
	precision	recall	f1-score	support
0	0.86	0.87	0.87	1000
1	0.99	0.98	0.99	1000
2	0.87	0.85	0.86	1000
3	0.86	0.94	0.90	1000
4	0.88	0.82	0.85	1000
5	0.99	0.94	0.96	1000
6	0.73	0.74	0.74	1000
7	0.92	0.97	0.94	1000
8	0.99	0.99	0.99	1000
9	0.96	0.95	0.96	1000
accuracy			0.91	10000
macro avg	0.91	0.91	0.91	10000
weighted avg	0.91	0.91	0.91	10000

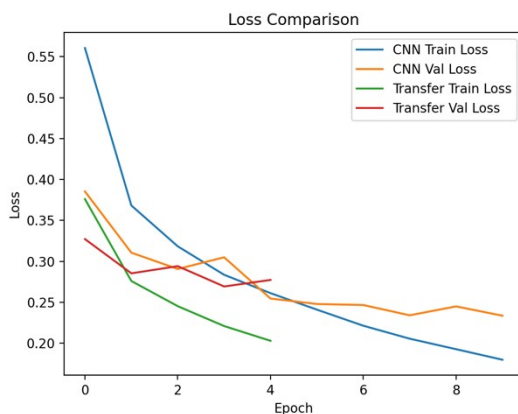
5.3 Computational Efficiency

Training time: The CNN took about 3 minutes to train 10 epochs on CPU, but the MobileNetV2 took about 2 minutes to train 10 epochs because it had fewer trainable parameters (frozen base layers). Nevertheless, the transfer model showed better generalisation performance, highlighting the efficiency and efficacy of transfer learning in terms of the computational efficiency and sufficiency in low-resource training requirements.



5.4 Error Analysis and Observations

The analysis of errors also highlights that false classifications were mostly due to ambiguous samples with visual data or similarity of category semantics. The CNN had minimal overfitting, which was reduced, through the use of dropout (dropout rate = 0.3) and early validation monitoring. The pretrained model, which enjoys the power of hierarchies of features, was able to generalise more with few training epochs, which further supports the applicability of pretrained structures to small and middle complex datasets like Fashion-MNIST.



6. Conclusion & Ethical Considerations

This paper explored the behaviour of two deep learning frameworks, one Convolutional Neural Network (CNN) that was built from scratch and the other using the transfer learning technique with MobileNetV2, on the Fashion-MNIST dataset. Empirical evidence showed that the MobileNetV2-based model was better than the baseline CNN in classification, generalization to unknown data, and training efficiency. This can be explained by the depthwise separable convolutions and trained feature extraction of MobileNetV2 on large-scale data (e.g. ImageNet), which facilitates strong representation learning even in low-resolution input.

Irrespective of these benefits, the research was limited in a number of ways. The grayscale resolution (28×28) limited the ability of the models to recognize finer visual detail, which may have been relevant to confuse the classes of semantically similar objects (e.g., “Shirt” vs. “T-shirt/top”). Additionally, the study shape of pretrained models was fixed, and thus, it required computationally intensive image resizing operations, which might cause spatial distortion.

Such models may be useful in fashion recommendation systems, inventory control, or automated tagging of e-commerce websites in real life. Nevertheless, morality is not a trivial issue [12]. Class imbalances or biased training data can lead to model bias, creating poorer performance for underrepresented fashion categories. Also, it can be misused during surveillance, where the classification of clothes may cross with the aspect of privacy or discriminatory profiling.

Future research might use more detailed datasets, add attention (e.g., CBAM or SE blocks), and expand the taxonomy to adopt more detailed fashion properties. Adding the interpretability techniques like Grad-CAM would also contribute to transparency and model credibility.

Reference:

- [1] Alateeq, M. M., PP, F. R., & Ali, M. A. (2023). Construction site hazards identification using deep learning and computer vision. *Sustainability*, 15(3), 2358.
- [2] Avazov, K., Jamil, M. K., Muminov, B., Abdusalomov, A. B., & Cho, Y. I. (2023). Fire detection and notification method in ship areas using deep learning and computer vision approaches. *Sensors*, 23(16), 7078.
- [3] Bouguettaya, A., Zarzour, H., Taberkit, A. M., & Kechida, A. (2022). A review on early wildfire detection from unmanned aerial vehicles using deep learning-based computer vision algorithms. *Signal Processing*, 190, 108309.
- [4] Chai, J., Zeng, H., Li, A., & Ngai, E. W. (2021). Deep learning in computer vision: A critical review of emerging techniques and application scenarios. *Machine Learning with Applications*, 6, 100134.
- [5] Dhanya, V. G., Subeesh, A., Kushwaha, N. L., Vishwakarma, D. K., Kumar, T. N., Ritika, G., & Singh, A. N. (2022). Deep learning based computer vision approaches for smart agricultural applications. *Artificial Intelligence in Agriculture*, 6, 211-229.
- [6] Drenkow, N., Sani, N., Shpitser, I., & Unberath, M. (2021). A systematic review of robustness in deep learning for computer vision: Mind the gap?. *arXiv preprint arXiv:2112.00639*.
- [7] Gaya-Morey, F. X., Manresa-Yee, C., & Buades-Rubio, J. M. (2024). Deep learning for computer vision based activity recognition and fall detection of the elderly: a systematic review. *arXiv preprint arXiv:2401.11790*.
- [8] Mettes, P., Ghadimi Atigh, M., Keller-Ressel, M., Gu, J., & Yeung, S. (2024). Hyperbolic deep learning in computer

vision: A survey. *International Journal of Computer Vision*, 132(9), 3484-3508.

[9] Oliveira, D. A. B., Pereira, L. G. R., Bresolin, T., Ferreira, R. E. P., & Dorea, J. R. R. (2021). A review of deep learning algorithms for computer vision systems in livestock. *Livestock Science*, 253, 104700.

[10] Paproki, A., Salvado, O., & Fookes, C. (2024). Synthetic data for deep learning in computer vision & medical imaging: A means to reduce data bias. *ACM Computing Surveys*, 56(11), 1-37.

[11] Steiniger, Y., Kraus, D., & Meisen, T. (2022). Survey on deep learning based computer vision for sonar imagery. *Engineering Applications of Artificial Intelligence*, 114, 105157.

[12] Yan, X., Zhang, H., Wu, Y., Lin, C., & Liu, S. (2023). Construction instance segmentation (CIS) dataset for deep learning-based computer vision. *Automation in Construction*, 156, 105083.

Appendix:

A. Dataset

The dataset used in this project is publicly available from Kaggle:

- Title: Fashion MNIST
- Source: <https://www.kaggle.com/datasets/zalando-research/fashionmnist>
- File Used: fashion-mnist_test.csv, fashion-mnist_train.csv.

B. Project Code

```

1  import pandas as pd
2  import numpy as np
3  import tensorflow as tf
4  import matplotlib.pyplot as plt
5  from sklearn.metrics import classification_report, confusion_matrix
6  import seaborn as sns
7
8  train_df = pd.read_csv("fashion-mnist_train.csv")
9  test_df = pd.read_csv("fashion-mnist_test.csv")
10
11 X_train = train_df.drop("label", axis=1).values.reshape(-1, 28, 28, 1) / 255.0
12 y_train = train_df["label"].values
13 X_test = test_df.drop("label", axis=1).values.reshape(-1, 28, 28, 1) / 255.0
14 y_test = test_df["label"].values
15
16 labels_map = {
17     0: "T-shirt", 1: "Trouser", 2: "Pullover", 3: "Dress", 4: "Coat",
18     5: "Sandal", 6: "Shirt", 7: "Sneaker", 8: "Bag", 9: "Ankle boot"
19 }
20
21 from tensorflow.keras import layers, models
22
23 cnn_model = models.Sequential([
24     layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
25     layers.MaxPooling2D(2, 2),
26     layers.Conv2D(64, (3, 3), activation='relu'),
27     layers.MaxPooling2D(2, 2),
28     layers.Flatten(),
29     layers.Dense(128, activation='relu'),
30     layers.Dropout(0.3),
31     layers.Dense(10, activation='softmax')
32 ])
33
34 cnn_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
35 cnn_history = cnn_model.fit(X_train, y_train, epochs=10, validation_split=0.1, batch_size=64)
36
37 from tensorflow.keras.applications import MobileNetV2
38 from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
39 from tensorflow.keras.models import Model
40

```

```

40
41 X_train_resize = np.repeat(tf.image.resize(X_train, (96, 96)).numpy(), 3, axis=-1)
42 X_test_resize = np.repeat(tf.image.resize(X_test, (96, 96)).numpy(), 3, axis=-1)
43
44 base_model = MobileNetV2(include_top=False, weights='imagenet', input_shape=(96, 96, 3))
45 base_model.trainable = False
46
47 x = GlobalAveragePooling2D()(base_model.output)
48 x = Dense(128, activation='relu')(x)
49 output = Dense(10, activation='softmax')(x)
50
51 transfer_model = Model(inputs=base_model.input, outputs=output)
52 transfer_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
53
54 transfer_history = transfer_model.fit(X_train_resize, y_train, epochs=5, validation_split=0.1, batch_size=64)
55
56 from tensorflow.keras.preprocessing.image import ImageDataGenerator
57
58 datagen = ImageDataGenerator(
59     rotation_range=15,
60     zoom_range=0.1,
61     width_shift_range=0.1,
62     height_shift_range=0.1
63 )
64 datagen.fit(X_train)
65
66 from tensorflow.keras.models import Sequential
67 from tensorflow.keras.layers import Dense, Reshape, LeakyReLU, Flatten
68
69 def build_generator():
70     model = Sequential([
71         Dense(256, input_dim=100),
72         LeakyReLU(alpha=0.2),
73         Dense(512),
74         LeakyReLU(alpha=0.2),
75         Dense(784, activation='tanh'),
76         Reshape((28, 28, 1))
77     ])
78     return model
79

```

```

80 def build_discriminator():
81     model = Sequential([
82         Flatten(input_shape=(28, 28, 1)),
83         Dense(512),
84         LeakyReLU(alpha=0.2),
85         Dense(256),
86         LeakyReLU(alpha=0.2),
87         Dense(1, activation='sigmoid')
88     ])
89     return model
90
91 def compile_gan(generator, discriminator):
92     discriminator.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
93     discriminator.trainable = False
94     gan = Sequential([generator, discriminator])
95     gan.compile(optimizer='adam', loss='binary_crossentropy')
96     return gan
97
98 generator = build_generator()
99 discriminator = build_discriminator()
100 gan = compile_gan(generator, discriminator)
101
102 for epoch in range(1000):
103     noise = np.random.normal(0, 1, (128, 100))
104     fake_images = generator.predict(noise)
105     real_images = X_train[np.random.randint(0, X_train.shape[0], 128)]
106
107     X_combined = np.concatenate([fake_images, real_images])
108     y_combined = np.concatenate([np.zeros((128, 1)), np.ones((128, 1))])
109
110     discriminator.trainable = True
111     discriminator.train_on_batch(X_combined, y_combined)
112
113     noise = np.random.normal(0, 1, (128, 100))
114     y_mislabeled = np.ones((128, 1))
115     discriminator.trainable = False
116     gan.train_on_batch(noise, y_mislabeled)
117
118 y_pred_cnn = cnn_model.predict(X_test).argmax(axis=1)
119 print("CNN Classification Report:\n", classification_report(y_test, y_pred_cnn))

```

```

119 print("CNN Classification Report:\n", classification_report(y_test, y_pred_cnn))
120
121 y_pred_transfer = transfer_model.predict(X_test_resize).argmax(axis=1)
122 print("Transfer Learning Report:\n", classification_report(y_test, y_pred_transfer))
123
124 cm = confusion_matrix(y_test, y_pred_cnn)
125 plt.figure(figsize=(8, 6))
126 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
127 plt.title("CNN Confusion Matrix")
128 plt.xlabel("Predicted")
129 plt.ylabel("Actual")
130 plt.show()
131
132 plt.plot(cnn_history.history['accuracy'], label='CNN Train Acc')
133 plt.plot(cnn_history.history['val_accuracy'], label='CNN Val Acc')
134 plt.plot(transfer_history.history['accuracy'], label='Transfer Train Acc')
135 plt.plot(transfer_history.history['val_accuracy'], label='Transfer Val Acc')
136 plt.title('Accuracy Comparison')
137 plt.xlabel('Epoch')
138 plt.ylabel('Accuracy')
139 plt.legend()
140 plt.show()
141
142 plt.plot(cnn_history.history['loss'], label='CNN Train Loss')
143 plt.plot(cnn_history.history['val_loss'], label='CNN Val Loss')
144 plt.plot(transfer_history.history['loss'], label='Transfer Train Loss')
145 plt.plot(transfer_history.history['val_loss'], label='Transfer Val Loss')
146 plt.title('Loss Comparison')
147 plt.xlabel('Epoch')
148 plt.ylabel('Loss')
149 plt.legend()
150 plt.show()

```